

GoldenGuide

Software Design Document

Basleal Abshiro

December 6, 2025

Contents

1 Project Overview and Goals **2**
1.1 Problem Statement . . . 2
1.2 Project Goal . . . 2
1.3 Core Objectives . . . 2

2 System Architecture **2**
2.1 High-Level Design Pattern: RAG . . . 2
2.2 Technical Components . . . 3
2.2.1 1. The Data Scraping Pipeline (Offline) . . . 3
2.2.2 2. The Vector Store (FAISS) . . . 3
2.2.3 3. The Response Layer (Online) . . . 3

3 Theoretical Framework: RAG Mechanics **4**
3.1 Semantic Search vs. Keyword Search . . . 4
3.2 Text Chunking . . . 4
3.3 Vector Embeddings . . . 4
3.4 FAISS and Indexing . . . 4

1 Project Overview and Goals

1.1 Problem Statement

The current help assistant `HawkBot` provided by Wilfrid Laurier University (WLU) is outdated and functionally limited. It lacks modern Artificial Intelligence capabilities seen everywhere, and makes asking specific questions either long or not available out of the pre-determined dialog options to pick from.

1.2 Project Goal

The goal of **GoldenGuide** is to engineer a next-generation academic assistant. Unlike the legacy system, GoldenGuide utilizes advanced Large Language Models (LLMs) to understand and answer specific questions. Crucially, the system is designed to provide **source links** for every claim it makes. This allows students to verify information directly on the university website, ensuring authenticity, correctness, and trust.

1.3 Core Objectives

- **Contextual Understanding:** Use Generative AI to interpret complex student questions.
- **Verifiability:** Implement a Retrieval-Augmented Generation (RAG) pipeline that cites specific WLU webpages.
- **Scalability:** Design a backend that can ingest thousands of university pages and serve accurate answers in real-time.

2 System Architecture

2.1 High-Level Design Pattern: RAG

The core architecture of GoldenGuide is built upon the **Retrieval-Augmented Generation (RAG)** pattern. Standard Large Language Models (like ChatGPT or Gemini) are trained on vast amounts of general public data, but they lack knowledge of private or highly specific real-time data, such as the specific academic dates or housing policies of Wilfrid Laurier University.

RAG bridges this gap by combining two distinct processes:

1. **Retrieval:** Searching a private database of WLU content to find relevant facts.
2. **Generation:** Passing those facts to the LLM to write a natural language answer.

This approach ensures the AI acts as a "reasoning engine" rather than a knowledge base, significantly reducing hallucinations (made-up facts) and allowing us to cite sources explicitly.

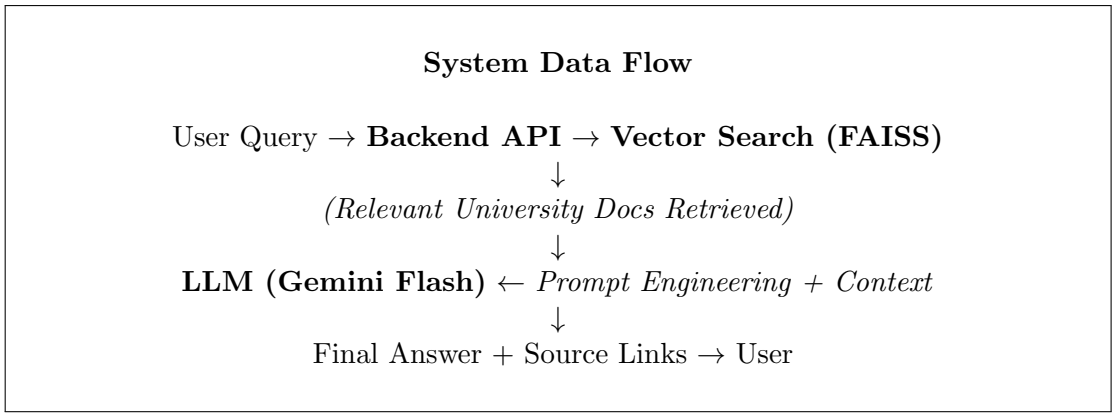


Figure 1: GoldenGuide High-Level Architecture Diagram

2.2 Technical Components

The system is composed of three primary technical layers, designed for modularity and speed.

2.2.1 1. The Data Scraping Pipeline (Offline)

Before the chatbot can answer questions, it must "read" the university website. This is an offline process that can be run periodically to update the knowledge base.

- **Scraper:** Written in Python using `BeautifulSoup`, this component parses the WLU `sitemap.xml` to ensure only valid, authoritative pages, can be scraped.
- **Cleaner:** Raw HTML is full of noise (scripts, navigation bars, etc). strip this away to leave only the core text content.
- **Chunker:** split long pages into smaller "chunks" (approx. 500 words). This is critical because if a page covers 20 different topics, we want to retrieve only the specific paragraph relevant to the student's question, not the entire page.

2.2.2 2. The Vector Store (FAISS)

To make the data searchable by "meaning" rather than just keywords, we convert text chunks into mathematical vectors using the `text-embedding-004` model provided by `Google AI`.

- We utilize **FAISS (Facebook AI Similarity Search)** as our vector database.
- **Why FAISS?** It is highly optimized for dense vector storage and allows for millisecond-latency searches, which is essential for a real-time chatbot experience.

2.2.3 3. The Response Layer (Online)

This is the live application that the student interacts with.

- **Frontend Interface:** implemented a lightweight, static HTML/JS client (`index.html`) to serve as the mock user interface. This demonstrates the system's modularity, allowing the chatbot logic to be accessed via standard REST calls (`fetch`) and demonstrates proof of concept without being tied to a complex frontend framework.
- **Backend API:** Built with **FastAPI**. This framework was chosen for its simplicity, ease of use, and quick testing of both `GET` and `POST` requests.
- **LLM Model:** We utilize Google's **Gemini Flash Latest**.

- **Why Flash?** For a student academic advising chatbot, speed is the mark. Flash offers lower latency than larger models while maintaining a large context window, allowing us to feed it dozens of university policy documents in a single prompt with more lax rate limits.

3 Theoretical Framework: RAG Mechanics

To understand how GoldenGuide achieves accuracy, it is necessary to detail the specific AI concepts employed: Text Chunking, Embeddings, and Vector Indexing.

3.1 Semantic Search vs. Keyword Search

Traditional search engines (like the legacy HawkBot) utilize **Keyword Search**. If a student asks "What is the cost of living?", the system looks for the exact word "cost". It may fail to retrieve a document that uses the word "fees" or "tuition" instead, even though the meaning is identical.

GoldenGuide utilizes **Semantic Search**. This allows the system to understand the *intent* and *meaning* behind words. It understands that "cost", "price", "fees", and "payment" are semantically related, ensuring relevant documents are found even if keywords do not exactly match.

3.2 Text Chunking

Large Language Models have a "Context Window", a limit on how much text they can process at once. Additionally, university webpages are often long and cover multiple topics.

To solve this, we implement **Chunking**.

- **Definition:** Breaking down long documents into smaller, self-contained segments (e.g., 500 words).
- **Rationale:** If a page discusses "Parking", "Tuition", and "Housing", and a student asks about "Parking", we do not want to retrieve the entire page. By chunking, we can retrieve *only* the paragraph relevant to parking. This increases the precision of the answer and reduces noise for the AI.

3.3 Vector Embeddings

Computers cannot inherently understand language; they understand numbers. To perform semantic search, we must translate text into a format the machine can calculate.

- **The Process:** We pass every text chunk through an embedding model via Google AI's `text-embedding-004`. This model outputs a **Vector**, a list of floating-point numbers (e.g., $[0.12, -0.5, 0.99, ...]$). In our system, each chunk is represented by a vector of 768 dimensions (`recommended dimension by google`).
- **Concept:** Imagine a 3D graph. Concepts that are similar in meaning (e.g., "Cat" and "Kitten") are placed physically close together on this graph. Concepts that are unrelated (e.g., "Cat" and "Calculus") are placed far apart.

3.4 FAISS and Indexing

Once we have thousands of vectors, searching them one by one is too slow for a real-time chatbot. This is where **FAISS (Facebook AI Similarity Search)** is utilized.

- **Indexing:** FAISS organizes our vectors into an efficient data structure (an index). Instead of checking every document, it can quickly narrow down the search to the relevant "neighborhood" of vectors.
- **Retrieval (L2 Distance):** When a user asks a question, we convert their question into a vector. FAISS then calculates the Euclidean distance (L2) between the question vector and our document vectors.
- **Top-K Retrieval:** We configure the system to retrieve the "Top-10" closest vectors. These represent the 10 most relevant pieces of information regarding the student's query.